

State Based Models!

- Search Problems
- Markov Decision Problems
- Adversarial Games

Search Problems:-

① Ex: Route Finding

Objective: Fastest?

Shortest?

Scenic?

Actions: Go fwd, turn left, turn right, etc.

Sol: Sequence of actions.

→ Current action affects future actions.

→ One wrong action can lead to consequences.

② Ex: Robot Motion Planning

Objective: Fastest?

Safest?

Energy-efficient?

Expressive way?

Actions: Rotate, Translate, etc.

③ <u>Ex</u> :	Famer	River	
	Cabbage	Boat	
	Wolf		
	Goat		

?

Approach: Tree Search

build a search tree "what if"

Actions:-

$F \rightarrow$ $F \leftarrow$ (cost 1 of each action)
 $FC \rightarrow$ $FC \leftarrow$
 $FG \rightarrow$ $FG \leftarrow$
 $FW \rightarrow$ $FW \leftarrow$

Sol:-

$FG \rightarrow, F \leftarrow, FC \rightarrow, FG \leftarrow, FW \rightarrow,$
 $F \leftarrow, FG \rightarrow$ (7 crosses)

Search Problem

$x \rightarrow f \rightarrow \text{seq. of actions}$

one action has
effect on future
actions.

VS Reflex Models

$x \rightarrow f \rightarrow y \in \{-1, +1\}$
one action

independent
at each step.

Formal Definition:-

- $S_{\text{start}} \rightarrow$ Start State
- $\text{Actions}_{(s)} \rightarrow$ Possible actions
- $\text{Cost}_{(s,a)} \rightarrow$ Action Cost
- $\text{Succ}_{(s,a)} \rightarrow$ Successor State (after action is performed)
- $\text{IsEnd}_{(s)} \rightarrow$ Reached the end? (bool)

Ex: Transportation Problem

- Street with $1-n$ blocks.
 - Walking from S to $S+1$ takes 1 min.
 - Taking a magic Tram from S to $2S$ is 2 min.
- How to travel from 1 to n in least time?

Constraints

1 se magic tram li tou cost zyada
End pe li tou street se bahir.

Traversal Algo.

- 1) Backtracking
- 2) DFS (Depth First Search)
- 3) BFS (Breadth First Search)
- 4) DLS

Fibonacci $\rightarrow$ fib-bo-na-chi

AP

Search Problems

- Start state
- possible Actions
- Action Cost
- Successor state
- Reached End State?

Algorithms:- (Tree Search)

① Backtracking Search

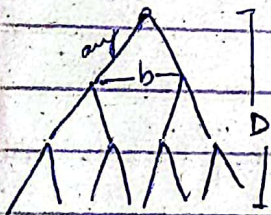
$\text{cost}(s, a) = \text{any}$ (no condition)

b : tree $\rightarrow$ branching factor $\rightarrow$ actions on each step.

D : max depth of Sol.

Space Complexity :- $O(D)$

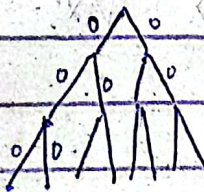
Time Complexity :- $O(b^D)$



$\rightarrow$ tree ko throughout search karna hai
har subat.

② Depth First Search

Assumption $\text{cost}(s, a) = 0$



$\rightarrow$ pehla best solution milte,
stop search. cz cost sb
hi zero hi hoge.

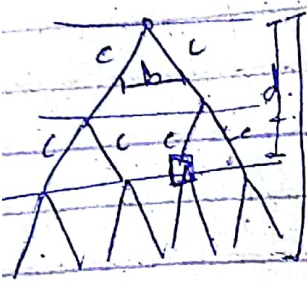
Saves Time

Space: $O(D)$ Time: $O(b^D)$

Problem with all these - Exponential Time Complexity.

③ Breadth First Search:-

Assume $\text{Cost}(s, a) = c$ where $c \geq 0$.



Space: $O(b^D)$

Time: $O(b^d)$ $d \ll D$

$\Rightarrow$ (Time efficient)

We want efficiency for Space and Time both.

④ Iterative Deepening Search:-

DPS and BFS combined:

$\rightarrow$ Each node at each level.

Go deeper on every iteration.

Space: $O(D)$

Time: $O(b^d)$ $d \ll D$

Summary:-

	Cost	Space	Time	
Backtracking	any	$O(D)$	$O(b^D)$	Cost
DFS	0	$O(D)$	$O(b^D)$	Shoe
BFS	$c \geq 0$	$O(b^D)$	$O(b^D)$	
DFS with Iterative deepening	$c \geq 0$	$O(D)$	$O(b^D)$	

Divide and Conquer VS Dynamic Programming

does not overlap subproblems.

→ solves them again & again

overlaps subproblems

→ solves once and stores result.

Dynamic Programming

To avoid exponential Time Complex.

$$\boxed{S} \leftarrow \begin{cases} \text{Future Cost}(s) = \min_a \{ \text{cost}(s, a) + \text{Future Cost}(s') \} & \text{otherwise} \\ 0 & \text{if End}(s) \text{ is true} \end{cases}$$

$\downarrow a$
 $\boxed{S'}$

$\left\{ \begin{array}{l} \text{Future Cost}(s') \\ \text{depends on current state only} \\ \text{Not history!} \end{array} \right.$

$\downarrow$
 $\boxed{\text{end}}$

Example: Find min cost from City - 1

to City - N. Only moving forward.

cost from i to j is C_{ij} .

(b, D)
subproblem
(b, 0)

→ Dynamic Prog. is same subproblems! (Store result) -

- Graph defined by Actions (s) and $\text{succ}(s, a)$ is Acyclic.

→ Cyclic hoga to future cost nikal rhi me confuse ho jaye ga algo.

Key Idea of State:-

State is a Summary of all past actions sufficient to choose future actions optimally.

Example

Route finding

* Constraint:- Can't visit 3 odd states in a row



How to define State Space?

→ Option-1

State $S = (\text{prev city}, \text{current city})$

no. of possible

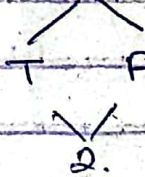
states $|S| = n \times n = n^2$

Constraint:-

odd cities $\rightarrow$ even cities

Option-2

$S = (\text{prev city odd?}, \text{current city})$



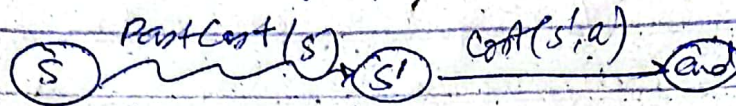
$|S| = 2n$

* Constraint:- At least 3 odd cities.

$S = (\min(\# \text{ of odd cities}, 3), \text{current city})$

Uniform Cost Search - (UCS)

Even for graphs with cycles.



In Order of increasing Path Cost!

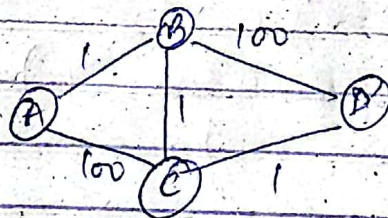
Assumption :- Non-Negativity!
 $\text{cost}(S, a) \geq 0$

Explored - States with optimal path found!

Frontier - Seen states, but figuring opt. path!

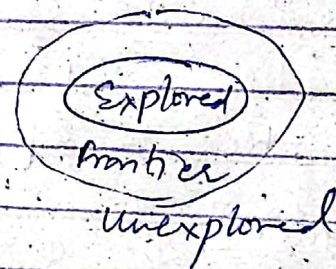
Unexplored - Unseen States

Example



Start $S = A$

End $S = D$



Unexp. States

Frontier

Explored

~~A~~
~~B~~
~~C~~
~~D~~

$A, 0 \rightarrow B, 1$
 $A, 0 \rightarrow C, 100$
 $B, 1 \rightarrow C, 1$
 $B, 1 \rightarrow D, 100$
 $C, 1 \rightarrow D, 1$

$A, 0$
 $B, 1$
 $C, 1$
 $D, 1$

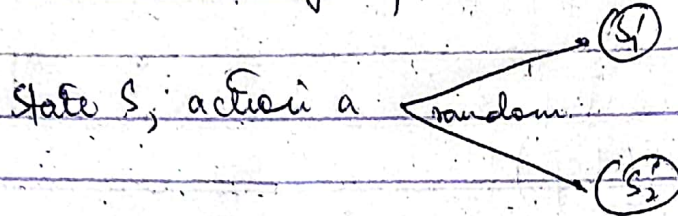
$A \rightarrow B \rightarrow C \rightarrow D$
 cost = 3

19/12/2018

A.I.

Markov Decision Process (MDPs)

Real World is highly uncertain.



Application:

- Robotics:

Decide where to move.

↳ actuators can fail

↳ can hit unseen obstacle.

- Resource Allocation:

Decide what to produce

↳ don't know demand

- Agriculture:

Decide what to plant

↳ weather uncertainty

Example:

Dice game

Each round:

- "stay" or "quit"

- If quit: take \$10 and end game.

- If stay: take \$4 and roll dice.

↳ If results in 1 or 2, end.

↳ otherwise, continue to next round.

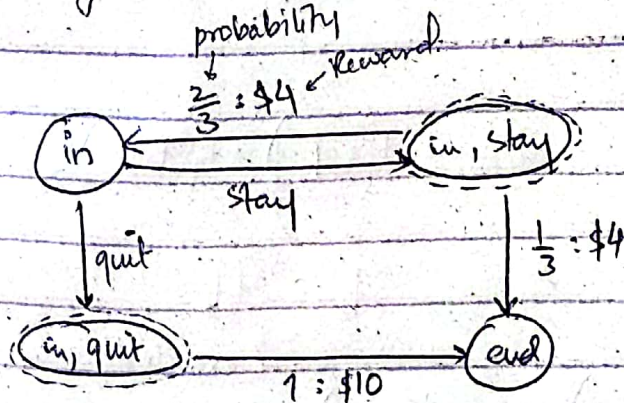
MDPs

- States: The set of states where s^* is start state
- Actions: Set of actions
- $T(s, a, s')$: transition probability of s' if take action a in state s .
- Reward (s, a, s') : Reward of transition $T(s, a, s')$
- $isEnd(s)$: whether end state or not?
- $0 \leq \gamma \leq 1$: discount factor (Default value = 1)
gamma

$$T(s, a, s') = \begin{cases} 1 & \text{succ}(s, a) = s' \\ 0 & \text{o/w} \rightarrow \text{otherwise} \end{cases}$$

MDP for Die Game

nodes $\rightarrow$ states
 nodes $\rightarrow$ chance nodes
 edges $\rightarrow$ actions



s	a	s'	$T(s, a, s')$
in	quit	end	1
in	stay	in	$2/3$
in	stay	end	$1/3$

Search Prob :- Sol: Seq. of actions (path)

MDPs :- Sol :- ~~Pol~~ Policy / Strategy.
 π

Policy: A policy π is mapping
from states ($s \in \text{states}$) to
actions ($a \in \text{actions}$).

$$\pi(s) \rightarrow A(s)$$

Policy Evaluation:

following a policy gives us a
random path.

Utility - Discounted Sum of Rewards
on the path.

random
variable
cr. of the
randomness!

$$U = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$$

actual reward value discount factor (0-1) design choice

Value (Expected Utility) $\{E(u)\}$

$E(u)$:- average of diff. utilities.

~~Value~~ $V_{\pi}(s)$ $\rightarrow$ (expected utility) value of policy π at state s .

Q-value :- $Q_{\pi}(s, a)$ (from chance nodes)

taking an action a then following π policy

22/12/23

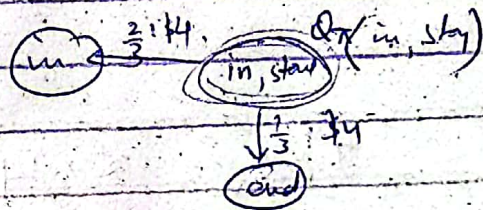
$$V_{\pi}(s) = \begin{cases} 0 & \text{if } \text{isEnd}(s) = T \\ Q_{\pi}(s, \pi(s)) & \text{otherwise} \end{cases}$$

$$\text{where } Q_{\pi}(s, \pi(s)) = \sum_{s'} T(s, \pi(s), s') \left[\underbrace{\text{Reward}(s, \pi(s), s')}_{\text{immediate reward}} + \underbrace{\gamma V_{\pi}(s')}_{\text{future reward}} \right]$$

Example -

$\pi(\text{in}) = \text{"stay"}$

$$V_{\pi}(\text{in}) = Q_{\pi}(\text{in}, \text{stay})$$



$$= \frac{1}{3} [4 + \gamma V_{\pi}(\text{end})] + \frac{2}{3} [4 + \gamma V_{\pi}(\text{in})]$$

Assume $\gamma = 1$

$$V_{\pi}(\text{in}) = \frac{4}{3} + \frac{2}{3} [4 + V_{\pi}(\text{in})] \quad \text{closed form solution}$$

$$V_{\pi}(\text{in}) = \frac{4}{3} + \frac{8}{3} + \frac{2}{3} V_{\pi}(\text{in})$$

$$V_{\pi}(\text{in}) - \frac{2}{3} V_{\pi}(\text{in}) = \frac{12}{3}$$

$$\frac{1}{3} V_{\pi}(\text{in}) = \frac{12}{3}$$

$$\boxed{V_{\pi}(\text{in}) = 12}$$

{ Iterative or Recursive }

Algorithm for Policy Evaluation

Iterative Algo :-

at Timestamp 0

initialize $V_{\pi}^{(0)}(s) = 0 \quad \forall s \in \text{states}$

for iterations $t = 1, 2, 3, \dots$ $t_{PE} \leftarrow$ total ~~steps~~ ^{iterations} at policy evaluation

for each states :-

$$V_{\pi}^{(t)}(s) = \sum_{s'} T(s, \pi(s), s') \left[\text{Reward}(s, \pi(s), s') + \gamma V_{\pi}^{(t-1)}(s') \right]$$

Time Complexity :- $O(t_{PE} \cdot |s| \cdot |s'|)$
: cubic.

t	$V_{\pi}^{(t)}$	end	in
---	-----------------	-----	----

0	$V_{\pi}^{(0)}$	0	0
---	-----------------	---	---

1	$V_{\pi}^{(1)}$	0	$\frac{1}{3} [4 + \cancel{V_{\pi}^{(0)}(\text{end})}] + \frac{2}{3} [4 + \cancel{V_{\pi}^{(0)}(\text{in})}]$ $= \frac{4}{3} + \frac{8}{3} = \frac{12}{3} = 4$
---	-----------------	---	---

2	$V_{\pi}^{(2)}$	0	$\frac{1}{3} [4 + \cancel{V_{\pi}^{(1)}(\text{end})}] + \frac{2}{3} [4 + \cancel{V_{\pi}^{(1)}(\text{in})}]$ $= \frac{4}{3} + \frac{2}{3}(8) = \frac{4}{3} + \frac{16}{3} = \frac{20}{3}$ $= 6.67$
---	-----------------	---	--

$\{ x \leftarrow 7+5; 4; \}$

So, $[x:L], [L \leftarrow 0] \vdash \{ x \leftarrow 7+5; 4; \} : \text{int}(4), [L \leftarrow 12]$
location initially zero (store)
context

So, $[x:L], [L \leftarrow 0] \vdash x \leftarrow 7+5 : 12, [L \leftarrow 12]$

$[L \leftarrow 0] [\frac{12}{L}] = [L \leftarrow 12]$

$\{, [x:L], [L \leftarrow 0] \vdash 7+5 : \text{int}(12), [L \leftarrow 0] \}$

$\{, [x:L], [L \leftarrow 0] \vdash 5 : \text{int}(5), [L \leftarrow 0] \}$

$\{, [x:L], [L \leftarrow 0] \vdash 7 : \text{int}(7), [L \leftarrow 0] \}$

QED

write
in
reverse
order
(down to
up)

Partial Correctness

Only results.

e.g. Operating System

you can't prove it.

termination, it is in

infinite loop of running

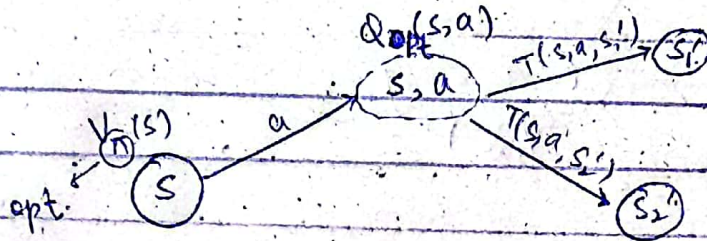
Total Correctness

Desired result +
termination of rules

Optimal Policy :- (π_{opt})

Find a policy to get $\max \cdot \Sigma(u)$.

Optimal Value :- max. value attained by any policy.



Objective - π_{opt} from V_{opt} .

$$Q_{opt}(S, a) = \sum_{S'} T(S, a, S') [R(S, a, S') - \gamma V_{opt}(S')]$$

$$V_{opt}(S) = \begin{cases} 0 & \text{if } \text{End}(S) = T \\ \max_{a \in \text{Actions}(S)} Q_{opt}(S, a) & \text{otherwise} \end{cases}$$

↳ actions that give Q_{opt} value maximizes reward value.

Given Q_{opt} , we can extract ' π_{opt} '

$$\pi_{opt}(S) = \arg \max_{a \in \text{Actions}(S)} Q_{opt}(S, a)$$

list of actions
as $\{a_1, a_2, \dots, a_n\}$

return argument
(give actions)

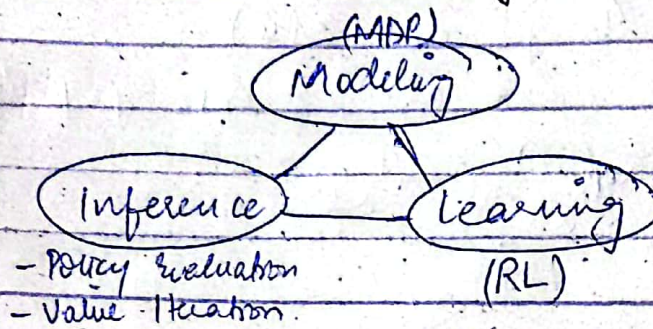
Also Output - Optimal Policy Value

Time Complexity :- $O(t_{vi} \cdot |S| \cdot |S'| \cdot |A|)$ → power 4

Convergence :- No further updates.

when $\gamma < 1$ or MDP graph is acyclic.

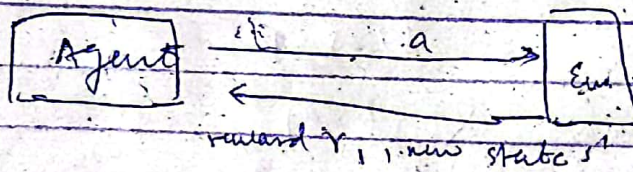
Reinforcement Learning



RL $\rightarrow$ parameters find ke li deta hai.

$\hookrightarrow$ we don't have Transition Probability and Reward ($T(s, a, s'), R(s, a, s')$)

Framework



• Monte Carlo

$\swarrow$ off policy

1- Model based Monte Carlo.

$\hookrightarrow$ Input data: $s, a, r, s', \dots, a, r, s'$
Est. $T(s, a, s')$ and $R(s, a, s')$

$$\hat{T}(s, a, s') = \frac{\# \text{ times } (s, a, s') \text{ occurs}}{\# \text{ times } (s, a) \text{ occurs}}$$

$$\hat{R}(s, a, s') = r \text{ in } (s, a, r, s')$$

$\rightarrow$ It is data driven.

Updates on each iteration.

fault $\rightarrow$ one Policy might not cover all actions.
we need to explore.

On ~~Off~~-Policy Alloc.

• Model free Monte Carlo :-

whats the need of T s and R s, directly compute $Q_{opt}(s,a)$!

- Data following a policy π
- Utility at Time Stamp t

$$U_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

$$\begin{aligned} \epsilon_{p-1} &: \text{---} (s,a) \text{---} \rightarrow U_t \\ \epsilon_{p-2} &: \text{---} (s,a) \text{---} \rightarrow U_t \\ \epsilon_{p-3} &: \text{---} (s,a) \text{---} \rightarrow U_t \end{aligned}$$

$$\hat{Q}_{\pi}(s,a) = \text{avg } U_t \quad \text{where } s_{t-1} = s, a_t = a.$$

first time jahan mile ga!
uss k agay se reward
nikal lena hai.

Assume $\gamma=1$.

Running Avg:-

Equivalent Formula (Convex Combination)

on each (s,a,u)

$$\hat{Q}_{\pi}(s,a) = (1-\eta) \hat{Q}_{\pi}(s,a) + \eta u$$

$$\text{where } \eta = \frac{1}{1 + \text{No. of updates to } (s,a)}$$

(Initially sb zero hoga hai)

SGD like Ref formula

Stochastic
gradient
descent

$$\hat{Q}_{\pi}(s,a) = \hat{Q}_{\pi}(s,a) - \eta [\hat{Q}_{\pi}(s,a) - u]$$

prediction target
↓ ↓

05/01/2024.

AI

On-policy Algo. estimate the value of data generating policy
Off-policy Algo. " " " another policy.

• Bootstrapping Method:-

- reward and Q-value.

→ SARSA Algo. ← On-Policy Algo.

↳ On each (s, a, r, s', a')

$$\hat{Q}_{\pi}(sa) = (1-\gamma) \underbrace{\hat{Q}_{\pi}(s,a)}_{\text{prediction}} + \gamma \underbrace{[r + \hat{Q}_{\pi}(s',a')]}_{\text{Target}}$$

→ Q-Learning ← Off-Policy Algo.

- by Watkins and Dayan in 1992.

- estimates $\hat{Q}_{\text{opt}}(s,a)$
↳ optimal

On each (s, a, r, s', a')

$$\hat{Q}_{\text{opt}}(s,a) = (1-\gamma) \hat{Q}_{\text{opt}}(s,a) + \gamma [r + \max_{a \in \text{Actions}} \hat{Q}_{\text{opt}}(s',a)]$$

Which Exploration Policy π_{act} to use?

Strategy:-

- No exploration, only exploitation

- Attempt $\pi_{\text{act}}(s) = \arg \max_{\text{action}} \hat{Q}_{\text{opt}}(s,a)$

↳ too greedy.

- No exploitation, Only exploration

$\pi_{\text{act}}(s) = \text{Random from Actions}$

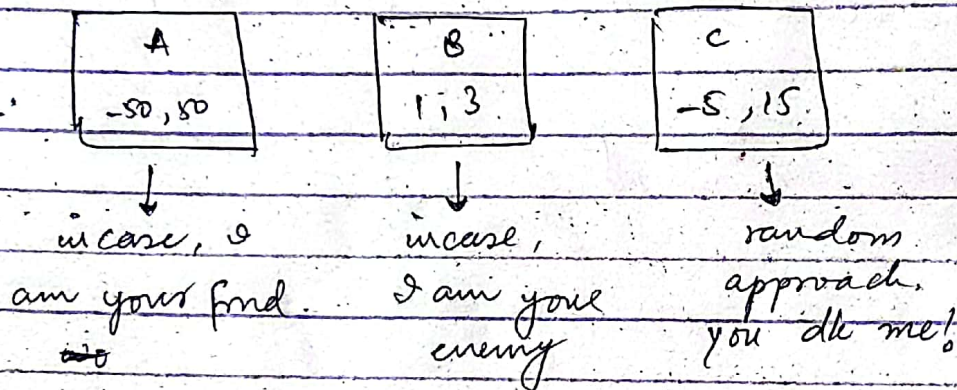
09/07/2024

AI

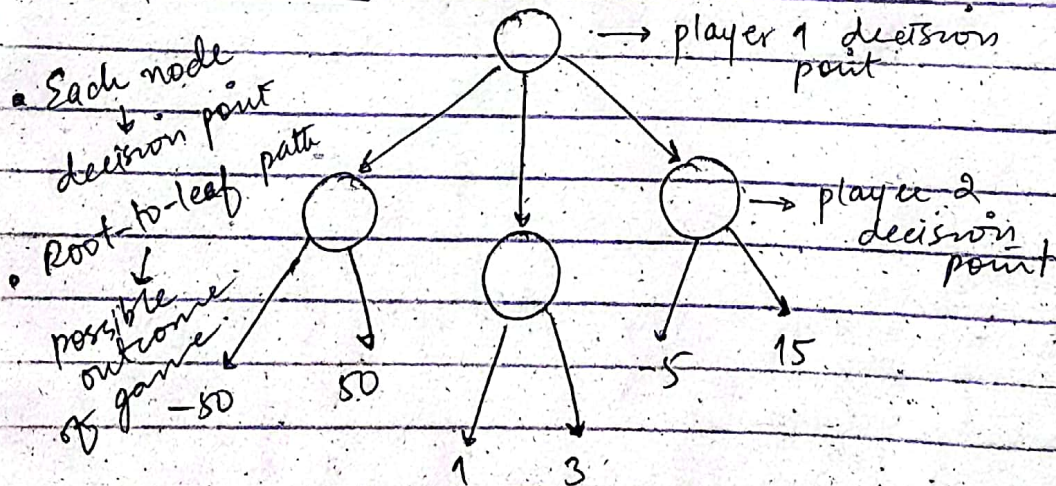
Adversarial Games:-

Example:-

- you choose a bin
- I choose a number from that bin
- Your goal is to maximize the chosen number.



Game Tree:-



Diff:-

- No reward on edges
- Ultimate reward → Win/Lose.

Two-player Zero-Sum Games

Def $\vdash$

S_{start} : start state

$\text{Actions}(s)$: possible actions from s

$\text{Succ}(s, a)$: resulting state if chosen action a in state s

$\text{isEnd}(s)$: s is end state (game over)

$\text{Utility}(s)$: agent's utility at end state s

$\text{Player}(s) \in \text{players}$: players who controls state s

Ex:-

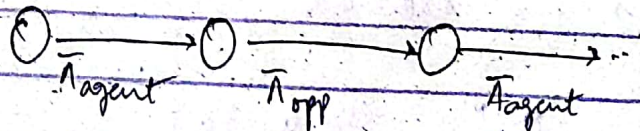
Chess

Game Evaluation

$$\pi_{\text{agent}}(s) = A$$

$$\pi_{\text{opp}}(s, a) = \frac{1}{2} \text{ for } a \in \text{Actions}(s)$$

G.E Recurrence



$$V_{\text{eval}}(s) = \begin{cases} \text{utility}(s) & \text{if } \text{isEnd}(s) = T \\ \max_{a \in \text{Actions}} \pi_{\text{agent}}(s, a) \cdot V_{\text{eval}}(\text{Succ}(s, a)) & \text{if agent} \\ \min_{a \in \text{Actions}} \pi_{\text{opp}}(s, a) \cdot V_{\text{eval}}(\text{Succ}(s, a)) & \text{if opp} \end{cases}$$

12/01/2024

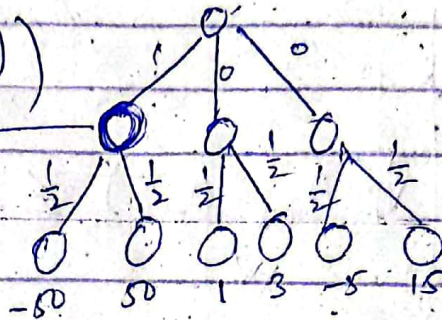
AI

Games Evaluation

(Recurrence)

$$V_{\text{eval}} = \sum_{\text{Actions}} \pi_{\text{opp}}(s, a) \cdot \text{Value}(\text{succ}(s, a))$$

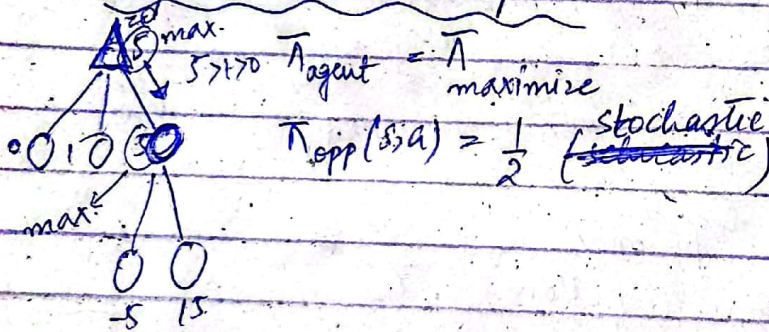
$$V_{\text{eval}} = \frac{1}{2}(-50) + \frac{1}{2}(50)$$



Goal:- Max. The chosen number.

Policy = ?

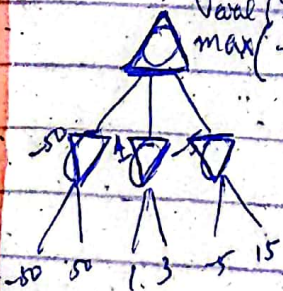
• Expectimax Example :-



• Minimax Example :- (worst enemy opponent)

$$\pi_{\text{agent}} = \pi_{\text{max}}, \quad \pi_{\text{opp}} = \pi_{\text{min}}$$

$$\text{Value}(s_{\text{start}}) = \max(-50, 1, -5)$$



Extract Policy:

(Value Iteration)

$$\pi_{\text{max}}(s) = \arg \max_{a \in \text{Actions}} V_{\text{minimax}}(\text{succ}(s, a))$$

$$\pi_{\text{min}}(s) = \arg \min_{a \in \text{Actions}} V_{\text{minimax}}(\text{succ}(s, a))$$

KEY

Δ max nodes

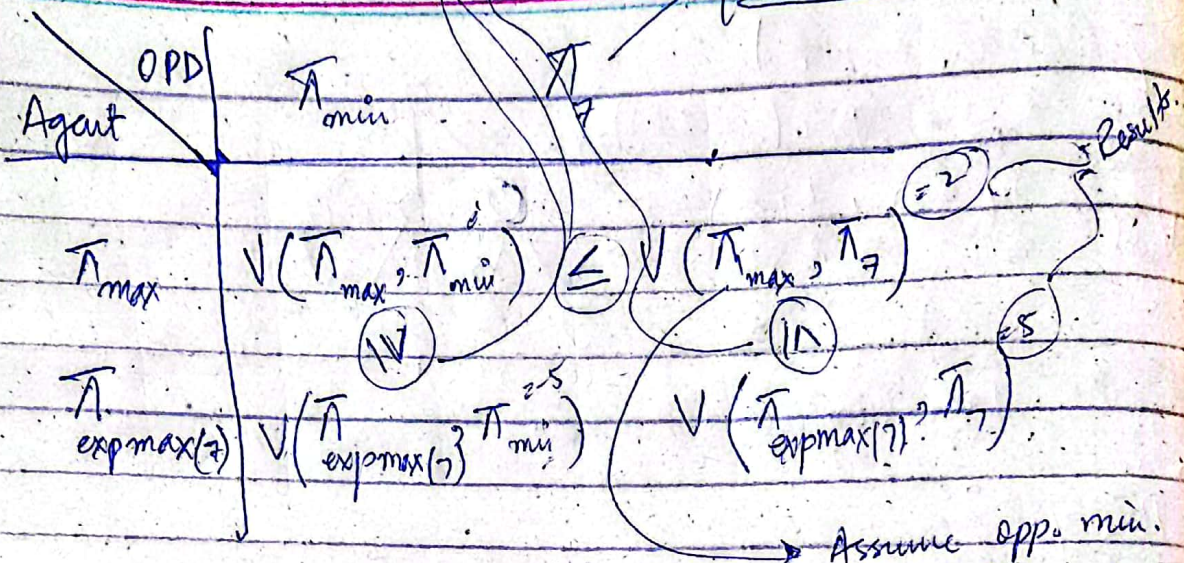
∇ min nodes

$\bigcirc$ stochastic

less than equals to.

Assumption

$$\pi_7(s, a) = \frac{1}{2}$$

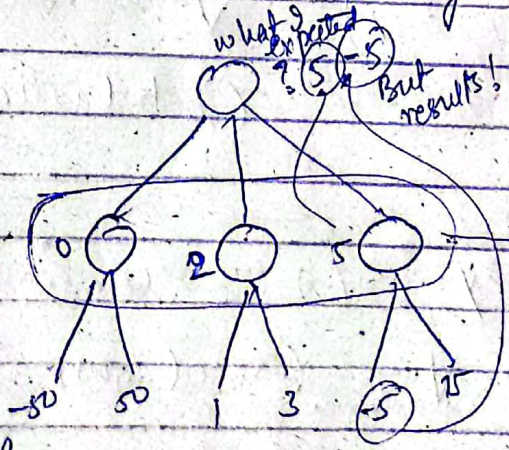


$$V(\pi_{\max}, \pi_{\min}) = 1 \quad (\text{prev. page})$$

$$V(\pi_{\text{expmax}(7)}, \pi_7) =$$

Assume opp. min. but π_7 has better results than expected. So, assume opp. to be min. always.

Agent is following $\pi_{\max}$ assuming opp. is following π_7 but actually opp. is minimizer.



Assuming opp. stochastic policy
avg value
0, 2, 5
I should choose this.

Key.

If you know opp. policy and opp. actually follows that policy, you can act accordingly, and gain max. utility.

But opp is minimizer it will return you -5

Alpha-beta pruning algo

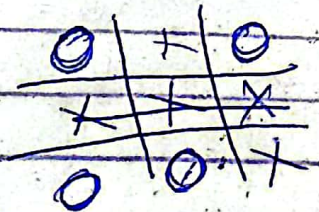
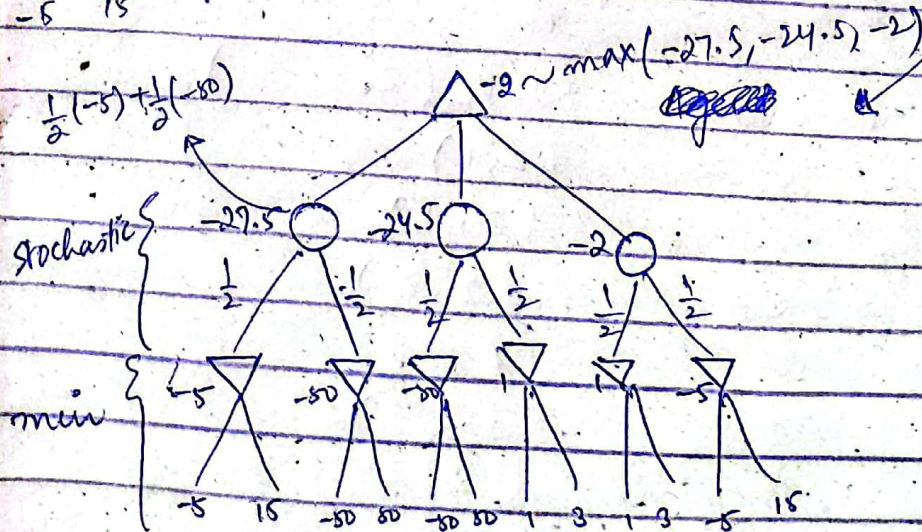
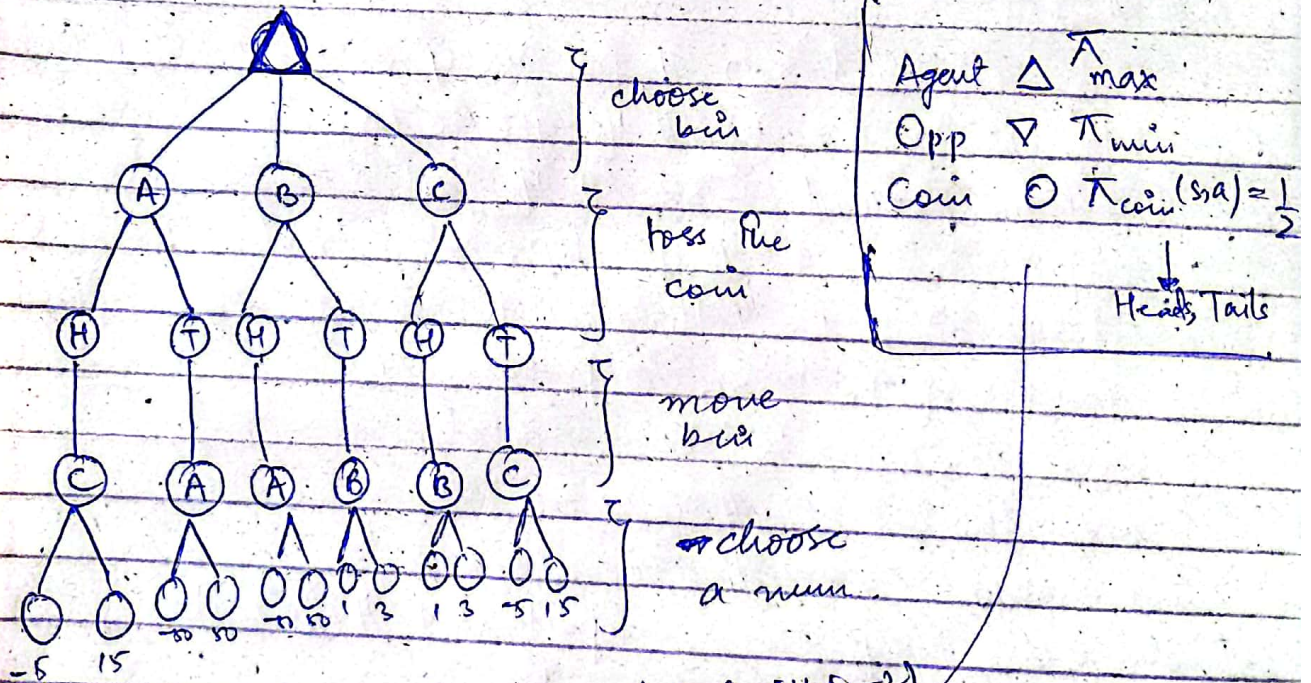
Modified Game:-

- You choose 1 of the 3 bins.
 - I toss a coin. if head, move 1 bin to the left.
 - I choose a no. from that bin. (wrap around)
- Goal: max the chosen number.

A
-50, 50

B
1, 3

C
-5, 15



19/07/2024

AI

Games Computation

- Game Tree

↳ $V_{\text{minimax}}(s)$

DFS:

Space : $O(d)$ → no. of players
Time : $O(b^d)$ → depth

e.g. Chess → $b \approx 35$, $d \approx 50$

Complex Computation

↳ then limited depth search.



Speedup minimax

Goal: Estimate $V_{\text{minimax}}(s)$

① Evaluation Funct. :- (Heuristic)

↳ use Domain Specific Knowledge
(Estimate of $V_{\text{minimax}}(s)$)

② Alpha beta pruning :-

↳ General purpose (Exact Answer)

ignores some branches
no need to explore whole tree.

→ 1 level down when → each player does a turn.

(asal mein tree pe 2 level neechay for railway head) → your turn, opp. turn.

Self defined formula: $Eval(s) = material + mobility + king\ safety + Centre$
 $100(K-K') + 90(Q-Q') + 9(B-B') + 7(N-N') + 1(P-P')$

most imp. king (highest weightage)

$$V_{\text{minimax}}(s, d) = \begin{cases} \text{Utility}(s) & \text{if } d = 0 \\ Eval(s) & \text{if } d = 0 \\ \max_{a \in \text{Actions}} V_{\text{minimax}}(\text{Succ}(s, a), d) & \text{if player = Agent} \\ \min_{a \in \text{Actions}} (V_{\text{minimax}}(a, d-1)) & \text{if player = opponent} \end{cases}$$

② Alpha beta pruning:-

Case-I

A	B
(3 ~ 5)	(5 ~ 100)
↑	↑

non-trivial overlap
(overlap at one point only)

So it can be pruned.

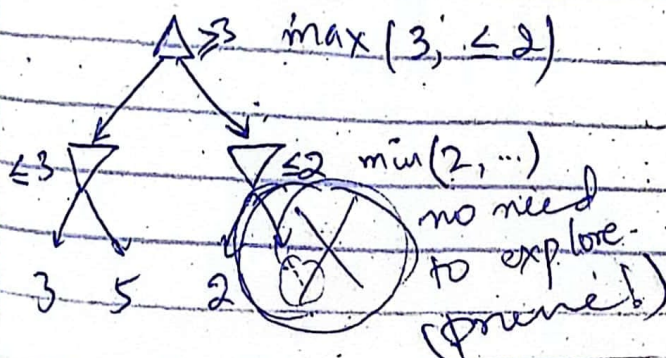
Goal:- max no. change.

So, we don't need to explore A.
Hence, it can be pruned.

Case-II

A	B
(3 ~ 5)	(4 ~ 10)

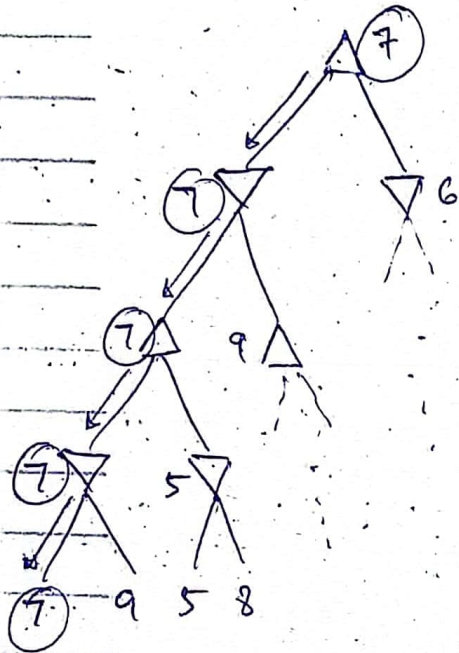
There's an overlap.
So, A can't be pruned.



Optimal Path:-

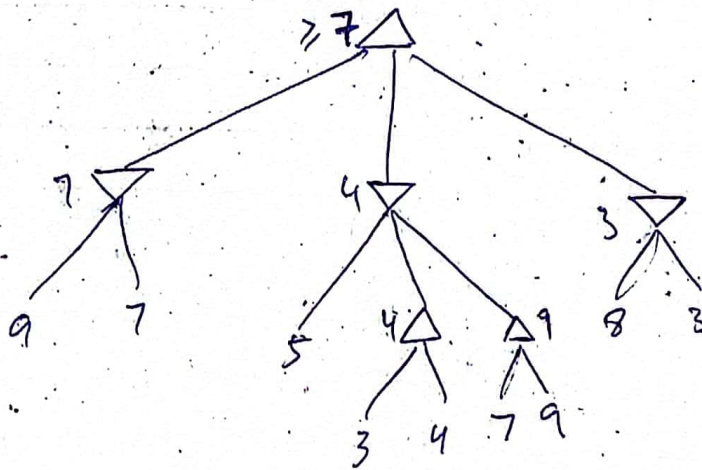
↳ path that minimax policies take!

↳ every node has same value!



↳ Optimal Path k ilawa-baki part ki pruning ho Sakti hai.

Alpha-beta pruning!



a_s = lower bound on max node s

b_s = upper " " " min "

α_s = $\max(a_s)$ for all s states.

β_s = $\min(b_s)$ for " " "

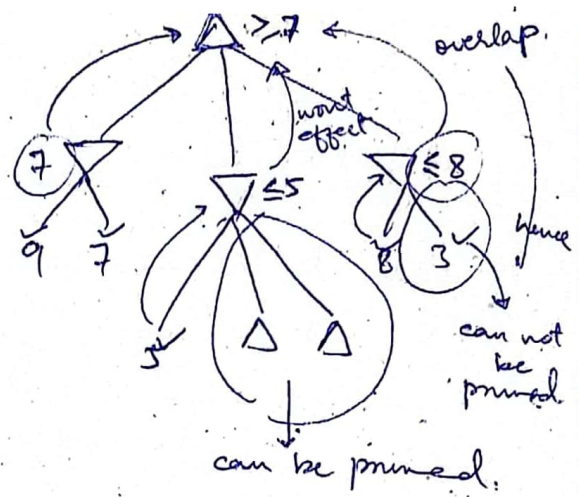
$$a_{s_1} = 7$$

$$\alpha_s = 7$$

$$b_{s_{22}} = 5$$

$$\beta_s = 5$$

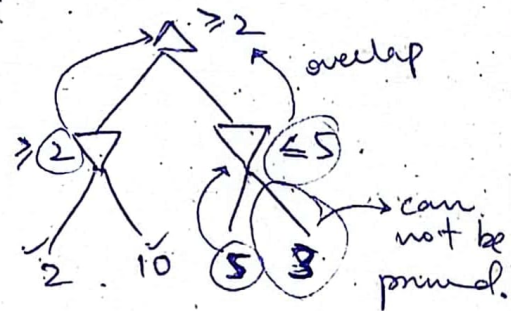
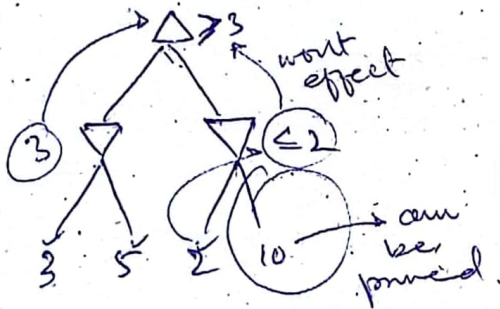
level node



The part that won't effect the root node can be pruned.

NOTE

Ordering is imp. in Pruning.



Worst Ordering

$$O(b^{2d}) \rightarrow \text{No pruning.}$$

Best Ordering

$$O(b^{2(0.5d)}) \rightarrow 50\% \text{ prune.}$$

Random Ordering

$$O(b^{2(0.75d)}) \rightarrow 25\% \text{ prune.}$$

max node pe $\rightarrow$ decreasing order
min node pe $\rightarrow$ increasing order

then can be pruned!

